Laporan Tugas Kecil

Strategi Algoritma (IF2211)

Disusun Oleh

Ahmad Rinofaros Muchtar (13524138)

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

BANDUNG 2025

1. Algoritma
    A. Memetakan grid sesuai dengan input, masing-masing dengan boolean false
    B. Secara iteratif algoritma memastikan tidak ada ratu di kolom yang sama dan bersebelahan secara horizontal atau diagonal
    C. Bila memenuhi syarat, update boolean menjadi true
    D. Apabila terjadi konflik di kolom berikut, update boolean sebelumnya menjadi false dan maju satu iterasi
    E. Apabila semua baris memiliki ratu, maka solusi valid ditemukan

2. Source Program

```
#include <iostream>
#include <vector>
#include <set>
#include <string>
#include <fstream>

#include <chrono>

using namespace std;

class solveQueens
{
private:
    vector<vector<char>> colorGrid;
    vector<vector<bool>> board;
    set<char> filledColors;
    long iterations = 0;
    int N;

public:
    solveQueens(vector<vector<char>> grid) : colorGrid(grid)
    {
        N = grid.size();                          // Menyimpan
dimensi NxN papan
        board.assign(N, vector<bool>(N, false)); // Membuat
papan NxN dengan boolean false(Tidak ada ratu)
    }

    bool isSafe(int row, int col)
    {
```

```
        char color = colorGrid[row][col];

        // Jika sudah ada warna dengan ratu, semua bagian papan
dengan warna itu mengembalikan false
        if (filledColors.count(color))
            return false;

        for (int i = 0; i < N; i++)
        {
            for (int j = 0; j < N; j++)
            {
                if (board[i][j])
                {
                    // Mengecek tidak ada ratu di kolom yang
sama
                    if (j == col)
                        return false;
                    // Mengecek bahwa ratu tidak bersebelahan
secara horizontal atau diagonal
                    if (abs(i - row) <= 1 && abs(j - col) <= 1)
                        return false;
                }
            }
        }
        return true;
    }

    // Algoritma bruteforce tanpa heuristik
    bool solve(int row)
    {
        if (row == N) // Jika semua baris memiliki ratu, maka
solusi valid ditemukan
            return true;

        for (int col = 0; col < N; col++)
        {
            iterations++; // Untuk menghitung setiap permutasi
yang telah dicek

            if (isSafe(row, col))
            {
```

```
                board[row][col] = true;

filledColors.insert(colorGrid[row][col]);//Mengupdate status
true pada warna yang sebelumnya belum memiliki ratu
                printBoard();
                for (int i = 0; i < N; i++)
                    cout << ".";
                cout << "\n";

                if (solve(row + 1))
                    return true;

                // Saya baru sadar bahwa tidak seharusnya
memakai backtrack karena termasuk heuristik, namun saya sudah
terlanjur
                board[row][col] = false;
                filledColors.erase(colorGrid[row][col]);//Jika
baris berikutnya gagal menemukan solusi, hapus kembali ratu
sebelumnya dan mencoba kolom berikutnya
            }
        }
        return false;
    }

    void printBoard()
    {
        for (int i = 0; i < N; i++)
        {
            for (int j = 0; j < N; j++)
            {
                if (board[i][j])
                    cout << "#";
                else
                    cout << colorGrid[i][j];
            }
            cout << endl;
        }
    }

    long long getIterations() { return iterations; }
    vector<vector<bool>> getBoard() { return board; }
};
```

```
vector<vector<char>> readBoard(string filename)
{
    ifstream file(filename);
    vector<vector<char>> grid;
    string line;
    while (getline(file, line))
    {
        if (line.empty())
            continue;
        grid.push_back(vector<char>(line.begin(), line.end()));
    }
    return grid;
}

void saveOutput(int N, const vector<vector<char>> &colorGrid,
const vector<vector<bool>> &board)
{
    string outFileName;
    cout << "Masukkan nama file untuk menyimpan solusi (contoh:
output.txt): ";
    cin >> outFileName;

    ofstream outFile(outFileName);
    if (outFile.is_open())
    {
        for (int i = 0; i < N; i++)
        {
            for (int j = 0; j < N; j++)
            {
                if (board[i][j])
                    outFile << "#";
                else
                    outFile << colorGrid[i][j];
            }
            outFile << "\n";
        }
        outFile.close();
        cout << "Solusi berhasil disimpan ke " << outFileName <<
endl;
    }
    else
```

```
    {
        cout << "Gagal membuat file output." << endl;
    }
}

int main()
{
    string filename;
    cout << "Pilih file test case (.txt): ";
    cin >> filename;

    vector<vector<char>> grid = readBoard(filename);
    if (grid.empty())
    {
        cout << "File tidak ditemukan atau kosong!" << endl;
        return 1;
    }

    solveQueens solver(grid);

    auto start = chrono::high_resolution_clock::now();
    bool found = solver.solve(0);
    auto end = chrono::high_resolution_clock::now();

    auto duration =
chrono::duration_cast<chrono::milliseconds>(end - start);

    if (found)
    {
        for (int i = 0; i < grid.size(); i++)
            cout << "*";
        cout << "\n";
        solver.printBoard();
        cout << "Waktu pencarian: " << duration.count() << " ms"
<< endl;
        cout << "Banyak kasus yang ditinjau: " <<
solver.getIterations() << " kasus" << endl;

        cout
            << "Apakah Anda ingin menyimpan solusi? (Ya/Tidak):
";
        string choice;
```

```
        cin >> choice;

        if (choice == "Ya" || choice == "ya")
        {
            saveOutput(grid.size(), grid, solver.getBoard());
        }

        else
        {
            solver.printBoard();
            cout << "Waktu pencarian: " << duration.count() << "
ms" << endl;
            cout << "Banyak kasus yang ditinjau: " <<
solver.getIterations() << " kasus" << endl;
            cout << "Tidak ada solusi yang valid." << endl;
        }

        return 0;
    }
}
```

3. Tangkapan Layar

A.Input

1. Contoh 1

```
AAABBCCCD
ABBBBCECD
ABBBDCECD
AAABDCCCD
BBBBDDDDD
FGGGDDHDD
FGIGDDHDD
FGIGDDHDD
FGGGDDHHH
```

2. Contoh 2

```
AAAAAAAAA
ABBBBCECD
ABBBDCECD
AAABDCCCD
ABBBDDDDD
AGGGDDHDD
AGIGDDHDD
AGIGDDHDD
AGGGDDHHH
```

3. Contoh 3

```
ABBCDDDE
AABCCDDE
FAAAADDE
FFAAAADE
GGAAAADA
HGAAAAAA
HHHAAAAA
HHHHHAAA
```

4. Contoh 4

```
AAAAAAAAA
AAAAAAAAA
AAAAAAAAA
AAAAAAAAA
AAAAAAAAA
AAAAAAAAA
AAAAAAAAA
AAAAAAAAA
```

5. Contoh 5

```
AAAAAAAAA
BBBBBBBBB
CCCCCCCCC
DDDDDDDDD
EEEEEEEEE
FFFFFFFFF
GGGGGGGGG
HHHHHHHHH
```

B.Output

1. Contoh 1

```
ABBB#CECD
ABBBDC#CD
A#ABDCCCD
BBBBD#DDD
FGG#DDHDD
#GIGDDHDD
FG#GDDHDD
FGGGDDHH#
Waktu pencarian: 54 ms
Banyak kasus yang ditinjau: 7
659 kasus
Apakah Anda ingin menyimpan s
olusi? (Ya/Tidak):
```

2. Contoh 2

```
AAAAAAAA#
ABBBBC#CD
ABBB#CECD
AAABDCC#D
ABB#DDDDD
A#GGDDHDD
AGIGDDHDD
AGIGDDHDD
AGGGDDHHH
.........
```

3. Contoh 3

```
AB#CDDDE
AABC#DDE
FAAAADD#
#FAAAADE
GGAAAA#A
H#AAAAAA
HHHAA#AA
HHH#HAAA
Waktu pencarian: 12 ms
Banyak kasus yang ditinjau: 2
388 kasus
Apakah Anda ingin menyimpan s
olusi? (Ya/Tidak):
```

4. Contoh 4

```
AAAAAAA#
AAAAAAAA
AAAAAAAA
AAAAAAAA
AAAAAAAA
AAAAAAAA
AAAAAAAA
AAAAAAAA
........
```

5. Contoh 5

```
#AAAAAAA
BB#BBBBB
CCCC#CCC
D#DDDDDD
EEEEE#EE
FFFFFFF#
GGG#GGGG
HHHHHH#H
Waktu pencarian: 0 ms
Banyak kasus yang ditinjau: 1
08 kasus
Apakah Anda ingin menyimpan s
olusi? (Ya/Tidak):
```

6. Gagal membaca

```
ucil1_13524138$ ./bin/Queens
Pilih file test case (.txt):
testcase6.txt
File tidak ditemukan atau kos
ong!
```

4. Link repository: https://github.com/BourneAgainSpicy/Tucil1_13524138

5. Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

Ahmad Rinofaros Muchtar

	Poin	Ya	Tidak
	Program berhasil di kompilasi tanpa kesalahan	✓	
	Program berhasil di jalankan	✓	
	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
	Program memiliki Graphical User Interface (GUI)		✓

	Program dapat menyimpan solusi dalam bentuk file gambar		✓